

```

    pending >= 1;
} while (pending);
}

```

This is the core mechanism associated with its processing. You can see that it looks for pending *softirqs* and executes them. The handler is then registered at run-time via *open_softirq()* which can take three parameters -- *softirq's* index, handler function and value for the data field.

Now let's come to *tasklets*. First, you need to remember that they have got nothing to do with tasks! As we discussed earlier, *tasklets* also work in a fashion similar to that of *softirqs*. But they have an easy-to-handle interface and extended locking rules. You will be making use of *tasklets* in most cases. Only in very rare cases (when very high-frequency and highly threaded requirements demand it) do we employ *softirqs*. *tasklets* are represented by two *softirqs*: *HI_SOFTIRQ* and *TASKLET_SOFTIRQ* (which are made to run after executing *HI_SOFTIRQ*-based ones). Since they are built on top of *softirqs*, the implementation looks almost similar.

tasklet_struct structure is used to represent the *tasklet* and is defined by:

```

struct tasklet_struct {
    struct tasklet_struct *next; /* next tasklet in the list */
    unsigned long state; /* state of the tasklet */
    atomic_t count; /* reference counter */
    void (*func)(unsigned long); /* tasklet handler function */
    unsigned long data; /* argument to the tasklet function */
};

```

You can employ this in your codes, and the good news is that you don't have to work with the old BH interface any more, as the developers have removed it completely.

Random number generator

Linux has implemented a strong random number generator, which is actually based on the PGP's random number generation method. The generator takes in the 'environmental noise' from device drivers and directs them to an entropy pool. We can access this pool using user and kernel modes. The generator is so effective that the outsider can never predict its value.

Those who know the working mechanism of PGP know the importance of these types of numbers in areas like cryptography. Another point that you need to take into account is that this generator produces true random numbers. They are different from the ones (pseudo-random) that you create using functions in C library—there the problem is that if you know one number in the series, you can guess any other number in the series.

Now let's take a dip in the 'pool' and the associated initiation:

```

static int trickle_thresh = INPUT_POOL_WORDS * 28;

static DEFINE_PER_CPU(int, trickle_count) = 0;

static struct poolinfo {
    int poolwords;
    int tap1, tap2, tap3, tap4, tap5;
} poolinfo_table[] = {
    /* x^128 + x^103 + x^76 + x^51 + x^25 + x + 1 -- 105 */
    {128, 103, 76, 51, 25, 1},
    /* x^32 + x^26 + x^20 + x^14 + x^7 + x + 1 -- 15 */
    {32, 26, 20, 14, 7, 1},
};

/* x^2048 + x^1638 + x^1231 + x^819 + x^411 + x + 1 -- 115 */
{2048, 1638, 1231, 819, 411, 1},

/* x^1024 + x^817 + x^615 + x^412 + x^204 + x + 1 -- 290 */
{1024, 817, 615, 412, 204, 1},

/* x^1024 + x^819 + x^616 + x^410 + x^207 + x^2 + 1 -- 115 */
{1024, 819, 616, 410, 207, 2},

/* x^512 + x^411 + x^308 + x^208 + x^104 + x + 1 -- 225 */
{512, 411, 308, 208, 104, 1},

/* x^512 + x^409 + x^307 + x^206 + x^102 + x^2 + 1 -- 95 */
{512, 409, 307, 206, 102, 2},
/* x^512 + x^409 + x^309 + x^205 + x^103 + x^2 + 1 -- 95 */
{512, 409, 309, 205, 103, 2},

/* x^256 + x^205 + x^195 + x^101 + x^52 + x + 1 -- 125 */
{256, 205, 195, 101, 52, 1},

/* x^128 + x^103 + x^78 + x^51 + x^27 + x^2 + 1 -- 70 */
{128, 103, 78, 51, 27, 2},

/* x^64 + x^52 + x^39 + x^26 + x^14 + x + 1 -- 15 */
{64, 52, 39, 26, 14, 1},
};

```

Here the true random number is fully independent of its generating function. Now, let's see how this is done. We know from thermodynamics (Physics!) that entropy is a measurement of disorder and randomness in a system. To represent the randomness in information, John von Neumann suggested the use of the term 'Shannon entropy' and Claude Shannon used it in his theory. Please read the Wikipedia entry on Shannon entropy at [en.wikipedia.org/wiki/Entropy_\(information_theory\)](http://en.wikipedia.org/wiki/Entropy_(information_theory)) for more details.

```
static int debug = 0;
```